Introduction to Node.js

The `node` Command

We can execute Node.js programs in the terminal by typing the `node` command, followed by the file name. The example command runs `app.js`, assuming there is a file with that name in the current working directory.

```
node app.js
```

Node.js REPL

Node.js comes with REPL, an abbreviation for read-eval-print loop. REPL contains three different states:

- a **read** state where it reads the input from a user,
- an **eval** state where it evaluates the user's input
- a **print** state where it prints out the evaluation to the console

After these states are finished, the REPL loops through these states repeatedly. REPL is useful as it gives back immediate feedback which can be used to perform calculations and develop code.

```
$ node
Welcome to Node.js v22.19.0.
> console.log("HI")
HI
```

Node.js Global Object

The Node.js environment has a `global` object that contains every Node-specific global property. The global object can be accessed by executing `console.log(global)`, or simply `global` in the terminal with the REPL running. In order to see just the keys, use `Object.keys(global)` instead. Since `global` is an object, new properties can be assigned to it via `global.name_of_property = 'value_of_property'`.

```
// access global within a script
console.log(global);

// add a new property to global
global.car = 'DeLorean';
```

Node.js Process Object

A process is the instance of a computer program that is being executed. Node has a global `process` object with useful properties. One of these properties is `.env`, which stores and controls information about the environment.

```
console.log(process.env.PWD); // Prints:
/path/where/terminal/command/executed

if (process.env.NODE_ENV === 'development')
{
  startDevelopmentServer();
  console.log('Testing! Testing! Does
everything work?');
} else if (process.env.NODE_ENV ===
'production') {
  startProductionServer();
}
```

Node.js process.argv

`process.argv` is a property that holds an array of the command-line values that were provided when the current process was initiated. The first element in the array is the absolute path to the Node installation, followed by the path to the file that was executed, and then any additional command-line arguments provided.

```
// Command line values: node web.js testing
several features
console.log(process.argv[2]); // 'testing'
will be printed
```

Node.js `process.memoryUsage()`

`process.memoryUsage()` is a method that can be used to return information on the CPU demands of the current process. Here, “heap” refers to a pool of computer memory, rather than the data structure of the same name.
Using `process.memoryUsage()` will return an object in a format like the example given.

Node.js Modules

In Node.js, files are called modules. Modularity is a technique where a single program has distinct parts, each providing a single piece of the overall functionality, like pieces of a puzzle coming together to complete a picture. `require()` is a function used to import one module into another.

```
const baseball = require('./babeRuth.js')
```

Node.js Core Modules

Node has several modules included within the environment to efficiently perform common tasks. These are known as the **core modules**. The core modules are defined within Node.js's source and are located in the `lib/` folder. A core module can be accessed by passing a string with the name of the module into the `require()` function.

```
const util = require('util');
```

Listing Node.js Core Modules

All Node.js core modules can be listed in the REPL using the `builtinModules` property of the `module` module. This is useful to verify if a module is maintained by Node.js or a third party.

The `console` Module

The Node.js `console` module exports a `global` console object offering similar functionality to the JavaScript console object used in the browser. This allows us to use `console.log()` and other familiar methods for debugging, just like we do in the browser. And since it is a global object, there's no need to require it into the file.

```
console.log('Hello Node!'); // Logs 'Hello Node!' to the terminal
```

`console.log()`

The `console.log()` method in Node.js outputs messages to the terminal, similar to `console.log()` in the browser. This can be useful for debugging purposes.

```
console.log('User found!'); // Logs 'User found!' to the terminal
```

The `os` Module

The Node.js `os` module can be used to get information about the computer and operating system on which a program is running. System architecture, network interfaces, the computer's hostname, and system uptime are a few examples of information that can be retrieved.

```
const os = require('os');

const systemInfo = {
  'Home Directory': os.homedir(),
  'Operating System': os.type(),
  'Last Reboot': os.uptime()
};
```

The `util` Module

The Node.js `util` module contains utility functions generally used for debugging. Common uses include runtime type checking with `types` and turning callback functions into promises with the `.promisify()` method.

```
// typical Node.js error-first callback function
function getUser (id, callback) {
  return setTimeout(() => {
    if (id === 5) {
      callback(null, { nickname: 'Teddy' });
    } else {
      callback(new Error('User not found'));
    }
  }, 1000);
}

function callback (error, user) {
  if (error) {
    console.error(error.message);
    process.exit(1);
  }
  console.log(`User found! Their nickname is: ${user.nickname}`);
}

// change the getUser function into promise using `util.promisify()`
const getUserPromise = util.promisify(getUser);

// now you're able to use then/catch or async/await syntax
getUserPromise(id)
  .then((user) => {
      console.log(`User found! Their nickname is: ${user.nickname}`);
  })
  .catch((error) => {
      console.log('User not found', error);
  });
```